OBSCURE // ARCHITECTURE NOTES

FOUNDER STUDY GUIDE • STAGE 1

Stage 1: New Tab Tracer Bullet — Executive Engineering Notes

A clean, concept-first study guide for understanding, defending, and reverse-engineering Obscure's New Tab subsystem.

TARGET SUBSYSTEM	CHROMIUM BASELINE	EXTERNAL DIFF SIZE	STAGE 2 PERSISTENCE
New Tab Page (Launchpad)	**Stable 155.0.8059.39**	**≤10 Lines (HK-01, HK-02)**	**Gated (ADR 0002 Proposed)**

01 The Mental Model: What is a Tracer Bullet?

The "Tracer Bullet" Principle — ARCHITECTURAL PATTERN

In complex software, a **Tracer Bullet** is not a prototype you throw away. It is the skinniest possible slice of production code that penetrates through *every architectural layer*—from the meta-build system to the UI window—proving that the plumbing works before building complex features.

In Stage 1, we do NOT build workspace persistence or database storage. We only prove that Obscure can compile custom C++, take control of `chrome://new-tab-page`, stream dark-mode Launchpad assets inside a sandboxed renderer, and strictly drop back to standard Chromium in Incognito mode.

PHASE 1 • BUILD HOOK
Ninja / GN Link
Chromium's build system compiles `//chrome/browser/obscure` alongside browser core.

PHASE 2 • NAVIGATION
Config Routing
`WebUIConfigMap` checks `ObscureNewTabPageUIConfig` when user opens a tab.

PHASE 3 • PRIVACY GATE
Incognito Guard
If browsing in Incognito/Guest, returns `false` to preserve privacy fallback.

PHASE 4 • SANDBOX DELIVERY
In-Memory Stream
WebUIDataSource streams Launchpad assets to sandboxed renderer (Seccomp: 2).

02 The "Shim Pattern": How We Avoid Rebase Hell

Why 99% of Chromium Forks Fail — LESSON IN ARCHITECTURE

Most developers who customize Chromium edit hundreds of upstream files (Omnibox, TabStrip, Profile). When Google releases a new Chromium stable milestone with 50,000 commits, their repository has thousands of merge conflicts and dies.

The Obscure Rule: Over 98% of our code lives in an isolated directory: `//chrome/browser/obscure/`. Upstream Chromium is touched in exactly **two files**, totaling under 10 lines of diff.

Hook ID	Target File	Diff Footprint	What It Does
HK-01	`chrome/browser/ui/webui/chrome_web_ui_configs.cc`	~3 lines (Line 333)	Substitutes `NewTabPageUIConfig` with `ObscureNewTabPageUIConfig`.
HK-02	`chrome/browser/BUILD.gn`	~2 lines (Line 1080)	Appends `"//chrome/browser/obscure"` to browser library dependencies.

Annotated Upstream Code Change (HK-01)

```
// Inside chrome_web_ui_configs.cc -> RegisterChromeWebUIConfigs() #if !BUILDFLAG(IS_ANDROID) // 1. Include our isolated header #include "chrome/browser/obscure/ui/webui/obscure_new_tab_page_ui_config.h" // 2. Swap the registration to Obscure's config map.AddWebUIConfig(std::make_unique<obscure::ObscureNewTabPageUIConfig>()); #endif
```

- `std::make_unique`: Heap-allocates our config object using single-ownership modern C++ semantics.
- `map.AddWebUIConfig(...)`: Registers the config in Chromium's internal URL lookup table for `chrome://new-tab-page`.

03 Component Breakdown: The 4 Native Files

1. ObscureNewTabPageUIConfig (The Gatekeeper)

PRIVACY INVARIANT

This class decides *when* Obscure is allowed to run. Its most critical function is `IsWebUIEnabled()`:

```
bool ObscureNewTabPageUIConfig::IsWebUIEnabled(content::BrowserContext* browser_context) { Profile* profile = Profile::FromBrowserContext(browser_context); // If Incognito or Guest, return false! Falls back to native Chromium private screens. return profile && !profile->IsOffTheRecord(); }
```

- **Why this matters:** Guarantees that Obscure custom UI and cached project workspaces **never appear in Incognito or Guest sessions** (TB-05, TB-06).

2. ObscureNewTabPageUI (The WebUI Controller)

DATA PIPELINE

The controller creates the local virtual server that feeds HTML, CSS, and JS to the renderer process:

```
// Inside ObscureNewTabPageUI constructor content::WebUIDataSource* source = content::WebUIDataSource::CreateAndAdd(context, chrome::kChromeUINewTabPageHost); // Streams compiled in-memory strings using RefCountedString source->SetRequestFilter( base::BindRepeating(&ObscureNewTabPageUI::ShouldHandleRequest), base::BindRepeating(&ObscureNewTabPageUI::HandleRequest)); // Enforce zero inline scripts via strict WebUI Content Security Policy source->OverrideContentSecurityPolicy( network::mojom::CSPDirectiveName::ScriptSrc, "script-src 'self' chrome://resources;");
```

- **Why `SetRequestFilter`?** Allows streaming compiled in-memory assets without modifying Chromium's complex `.pak` binary archives or resource headers. Keeps Stage 1 external diffs strictly ≤ 10 lines.
- **Why `script-src 'self'`?** Enforces that all JavaScript must be loaded from separate `.js` files via `<script type="module">`. Inline scripts are strictly blocked by the browser.

04 Security Model: The Multi-Process Sandbox Jail

Why Obscure is Far Safer Than Electron

SECURITY SUPERIORITY

In Electron, a renderer window often has direct access to Node.js APIs (e.g. `fs.readFile`, `child_process.exec`), making remote code execution trivial. In Obscure, our Launchpad runs inside a **heavily sandboxed Linux jail**:

Security Layer	Kernel Flag	What It Physically Enforces
Layer 1 Sandbox	Linux PID & User Namespaces	The renderer cannot see other operating system processes or access host directory paths.
Layer 2 Sandbox	`Seccomp: 2` (Syscall Filter)	Linux kernel whitelist. If compromised JS attempts unauthorized syscalls (e.g. `execve`), kernel kills process instantly.
Privilege Block	`NoNewPrivs: 1`	Guarantees that neither the process nor its children can ever escalate root privileges (e.g. via `sudo`).
WebUI Boundary	Site Isolation (`chrome://` scheme)	Standard internet web pages (`https://...`) are strictly forbidden from embedding or framing our internal WebUI.

05 Founder Mastery: Defending Obscure in 60 Seconds

Q: "Why modify Chromium C++ instead of building a Chrome Extension?"

Founder Answer: Chrome extensions are toys for cosmetic changes. An extension cannot control process memory limits, cannot isolate workspace networking, cannot inspect local listening ports safely, and can be disabled by Chrome policies at any time. By modifying Chromium's native C++ engine, Obscure controls the multi-process architecture directly while preserving full hardware-accelerated rendering and kernel sandboxing.

Q: "How do you know Google won't break your code on the next Chrome update?"

Founder Answer: Because we follow the *Shim Pattern*. Over 98% of Obscure code is confined to `//chrome/browser/obscure/`. Our external touchpoint in upstream Chromium is just two registration hooks totaling under 10 lines. When Chromium updates, we only have to re-verify those two hooks.

Q: "Does Obscure store workspace data on disk in Stage 1?"

Founder Answer: No. Stage 1 is strictly a Tracer Bullet. It proves the build pipeline, configuration routing, and sandboxed WebUI delivery. All native disk persistence and database serialization are intentionally deferred to Stage 2 under ADR 0002 to maintain rigorous scope boundaries.